Technical Documentation: RAG Support Assistant

1. Introduction

What is RAG: Retrieval-Augmented Generation (RAG) is a framework that improves Large Language Models (LLMs) by grounding their generation on external knowledge bases rather than relying solely on pre-trained weights.

Why it is needed: LLMs notoriously hallucinate facts or suffer from temporal knowledge cutoffs. By integrating a retrieval pipeline, this system injects exactly relevant snippets of custom internal documents directly into the prompt before generation, yielding high accuracy and perfect custom-domain compliance.

Use case overview: A robust support helpdesk interface allowing users to upload vast PDF-based knowledge bases (like technical manuals or billing policies). Users converse with an intelligent assistant that deflects common inquiries dynamically, escalating extremely complex or emotionally distressed questions directly to a human agent.

2. System Architecture Explanation

Detailed explanation of HLD: The system is designed functionally around a Streamlit UI communicating with a localized LangGraph orchestration engine. The orchestration layer drives a document injection pipeline (PyPDF -> Chunking -> HuggingFace embeddings -> ChromaDB vector persistence) and subsequently handles asynchronous conversational input.

Component interactions: A user query initially hits the Intent Router (an LLM categorization checkpoint). It subsequently trips either an Escalation side-effect, Clarification fallback, or standard Context Retrieval. The active retrieved context is evaluated securely via a dedicated LLM prompt guardrail before generation restricts output hallucinations.

3. Design Decisions

Chunk size choice: Configured structurally at 1000 characters with a 200 character overlap. This preserves maximum semantic value without prematurely severing technical context blocks.

Embedding strategy: Leveraged the HuggingFace `all-MiniLM-L6-v2`. Chosen for its extreme lightweight latency in generating 384-dimensional dense vectors efficiently on local CPU architecture, minimizing cloud dependency latency.

Retrieval approach: Scaled to Top_K=8 across isolated session indices natively in ChromaDB. Evaluates purely dense euclidean metrics across extensive manuals without dropping crucial metadata.

Prompt design logic: Implements robust zero-shot conditional logic with rigid LLM guards. e.g. explicitly mapping 'YES' / 'TRUE' Boolean evaluations to allow conversation fluff filtering out bad chunk associations autonomously.

4. Workflow Explanation

LangGraph usage: Unlike linear chains (LangChain Runnables), LangGraph fundamentally addresses branching and loopback cycles needed for realistic AI Agents.

Node responsibilities: Separate physical Python functions map entirely to 'detect_intent', 'retrieve_context', 'evaluate_context', 'generate_answer'. This compartmentalizes logic allowing flawless regression testing on isolated graph nodes.

State transitions: Implemented natively via a globally scoped 'GraphState' TypedDict (housing variables like retrieved_chunks, answer_draft, intent). Conditional routing evaluates mutations constantly over the dict.

5. Conditional Logic

Intent detection: Zero-shot classification directly against user queries targeting outputs to structured JSON (Answer, Clarify, Escalate).

Routing decisions: A conditional transition mapping edge values to functions natively. For instance, 'if route == answer: return retrieve_context'. It creates predictable pathways through unpredictable natural language.

6. HITL Implementation

Role of human intervention: Ensures AI boundaries are explicitly respected for complex billing, security, or unresolved confusion scenarios mapped to frustration intents.

Benefits and limitations: The asynchronous filesystem JSON ticketing is simple and doesn't throttle graph execution, yielding lightning fast response times. However, limiting synchronization to filesystem logic limits horizontal load-balancer scaling without migrating to a standard Postgres/Redis layer.

7. Challenges & Trade-offs

Retrieval accuracy vs speed: Increasing Top_K retrieval yields much higher contextual probability for semantic queries but proportionally increases LLM context parsing latency.

Chunk size vs context quality: Tiny chunks map Euclidean distance beautifully but yield extremely fractured contexts. 1000 characters was selected as the optimal compromise.

Cost vs performance: Transitioned to 'llama-3.1-8b-instant' via Groq to leverage native Language Processing Units (LPU), preserving incredibly aggressive generation speeds while maintaining robust parsing intelligence without hitting GPT-4 cost tiers.

8. Testing Strategy

Testing approach: Manual modular isolation. Injecting deterministic vectors and asserting predictable string output against 'evaluate_context' ensures prompt logic doesn't fracture.

Sample queries:
- Semantic ('What are data types in python?') -> Validated contextual bridging.
- Evaluative ('Summary of document') -> Validated prompt safety guardrails.
- Missing Data ('YOLO model') -> Validated strict failure without hallucination.

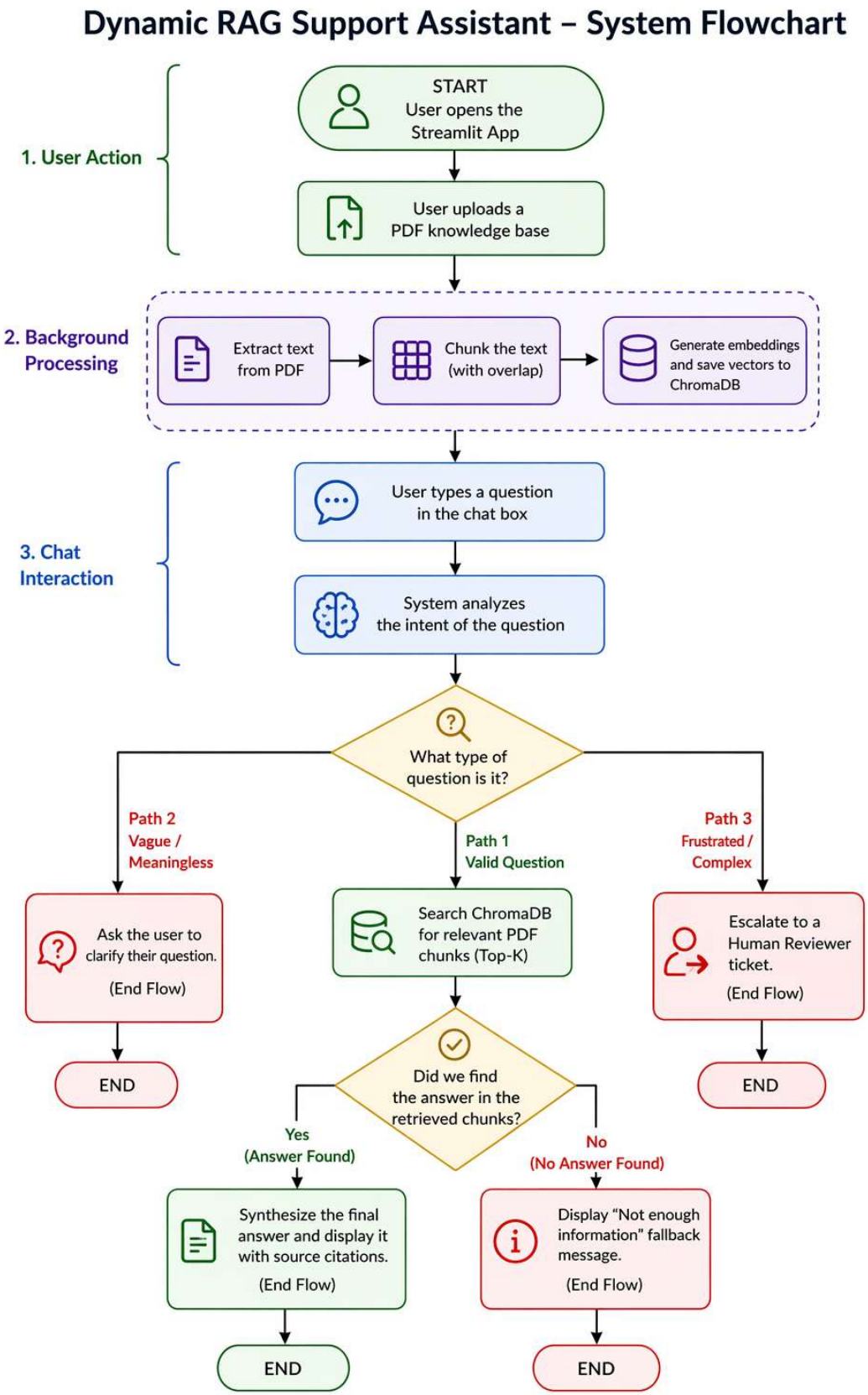
9. Future Enhancements

Multi-document support: Expanding mapping techniques beyond UI Session mapping to allow cross-system graph ingestion and vectorization.

Feedback loop: Adding thumb up/down UI interactions appending directly to a persistent database for LangSmith dataset optimizations.

Memory integration: Modifying `GraphState` strictly with standard LangChain native 'Memory' structures to retain conversational momentum over sequential turns.

Deployment: Refactoring Streamlit monolithic architecture strictly into decoupleable FastAPI endpoints natively communicating globally to React/Vercel dashboards.



FLOWCHART OF THE PROJECT